Ashwath Raj M | AP25110010712 | Core CSE A

GITHUB LINK: https://github.com/Ashwath-Raj/CSE-204-DAA-assignments

1. Implementation of Maximum and Minimum Using Divide and Conquer

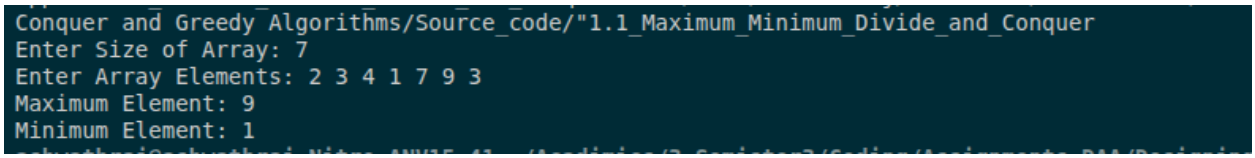
Problem

Write a program to find the maximum and minimum elements of an array using the divide and conquer strategy. Analyse and describe how the divide and conquer strategy is better when compared to the traditional approach.

Method

The array is recursively divided into two halves. The maximum and minimum values are found independently in the two halves and then combined. For two elements, the larger element is selected as maximum and the smaller as minimum.

Output:



Code:

```
/*
Program Name: Maximum and Minimum Using Divide and Conquer
Program Description: Takes an array as input and finds the maximum and minimum
                     elements using divide and conquer strategy
Program Creation Time: Sep 20th 4:30
Program Creator: Ashwath Raj | AP25110010712
*/

#include<iostream>
using namespace std;

struct Result{
    int max;
    int min;
};

Result findMaxMin(int arr[],int low,int high){
    Result r;

    if(low==high){
        r.max=r.min=arr[low];
        return r;
    }

    if(high==low+1){
        if(arr[low]>arr[high]){
            r.max=arr[low];
            r.min=arr[high];
        }
        else{
            r.max=arr[high];
            r.min=arr[low];
        }
        return r;
    }

    int mid=(low+high)/2;

    Result left=findMaxMin(arr,low,mid);
    Result right=findMaxMin(arr,mid+1,high);

    r.max=(left.max>right.max)?left.max:right.max;
    r.min=(left.min<right.min)?left.min:right.min;

    return r;
}

int main(void){
    int n;

    cout<<"Enter Size of Array: ";
    cin>>n;

    int arr[n];

    cout<<"Enter Array Elements: ";
    for(int i=0;i<n;i++)
        cin>>arr[i];

    Result r=findMaxMin(arr,0,n-1);
```

```
    cout<<"Maximum Element: "<<r.max<<endl;
    cout<<"Minimum Element: "<<r.min<<endl;

    return 0;
}
```

Complexity

Time Complexity: O(n)
Space Complexity: O(log n)

Comparison with Traditional Approach:
The traditional approach scans the array and maintains the current maximum and minimum. It requires 2n - 2 comparisons for n elements. The divide and conquer implementation compares pairs at the base cases and combines the results from the two halves. For even n, it requires about 3n/2 - 2 comparisons, so it reduces the number of comparisons while keeping the overall time complexity O(n). It also separates the problem into independent subproblems, which makes the strategy suitable for recursive and parallel implementations.

2. Divide and Conquer: Maximum Subarray Problem

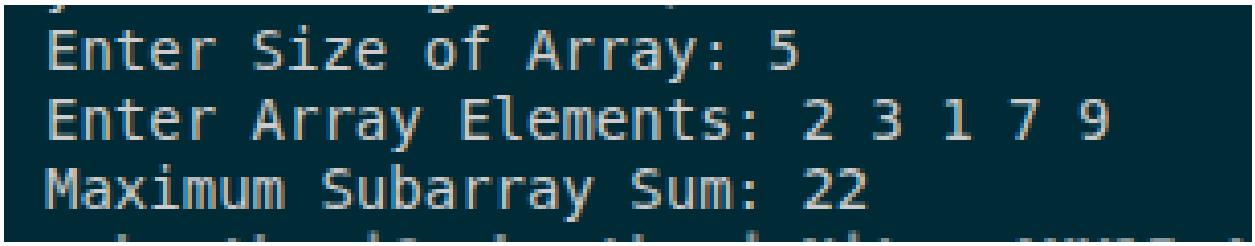
Problem

Write a program to solve the maximum subarray problem using the divide and conquer strategy.

Method

The array is divided into two halves. The maximum subarray is found in the left half, the right half, and the subarray crossing the middle. The largest of these three sums is returned.

Output:



Code:

```
/*
Program Name: Maximum Subarray Using Divide and Conquer
Program Description: Takes an integer array and finds the maximum possible
                     subarray sum using divide and conquer strategy
Program Creation Time: Sep 20th 4:40
Program Creator: Ashwath Raj | AP25110010712
*/

#include<iostream>
using namespace std;

int maxValue(int a,int b){
    return (a>b)?a:b;
}

int maxCrossingSum(int arr[],int low,int mid,int high){
    int sum=0;
    int leftSum=-1000000000;

    for(int i=mid;i>=low;i--){
        sum+=arr[i];
        if(sum>leftSum)
            leftSum=sum;
    }

    sum=0;
    int rightSum=-1000000000;

    for(int i=mid+1;i<=high;i++){
        sum+=arr[i];
        if(sum>rightSum)
            rightSum=sum;
    }

    return leftSum+rightSum;
}

int maxSubarray(int arr[],int low,int high){
    if(low==high)
        return arr[low];

    int mid=(low+high)/2;

    int left=maxSubarray(arr,low,mid);
    int right=maxSubarray(arr,mid+1,high);
    int cross=maxCrossingSum(arr,low,mid,high);

    return maxValue(maxValue(left,right),cross);
}

int main(void){
    int n;

    cout<<"Enter Size of Array: ";
    cin>>n;

    int arr[n];

    cout<<"Enter Array Elements: ";
    for(int i=0;i<n;i++)
        cin>>arr[i];

    cout<<"Maximum Subarray Sum: "<<maxSubarray(arr,0,n-1)<<endl;
    return 0;
```

```
}
```

Complexity

Time Complexity: O(n log n)
Space Complexity: O(log n)

Time Complexity Derivation:
At each recursive level, the array is divided into two subarrays and the crossing sum requires O(n) work. Therefore, the recurrence is T(n) = 2T(n/2) + O(n). With log n levels, the total time complexity is O(n log n).

3. Divide and Conquer: Strassen's Matrix Multiplication

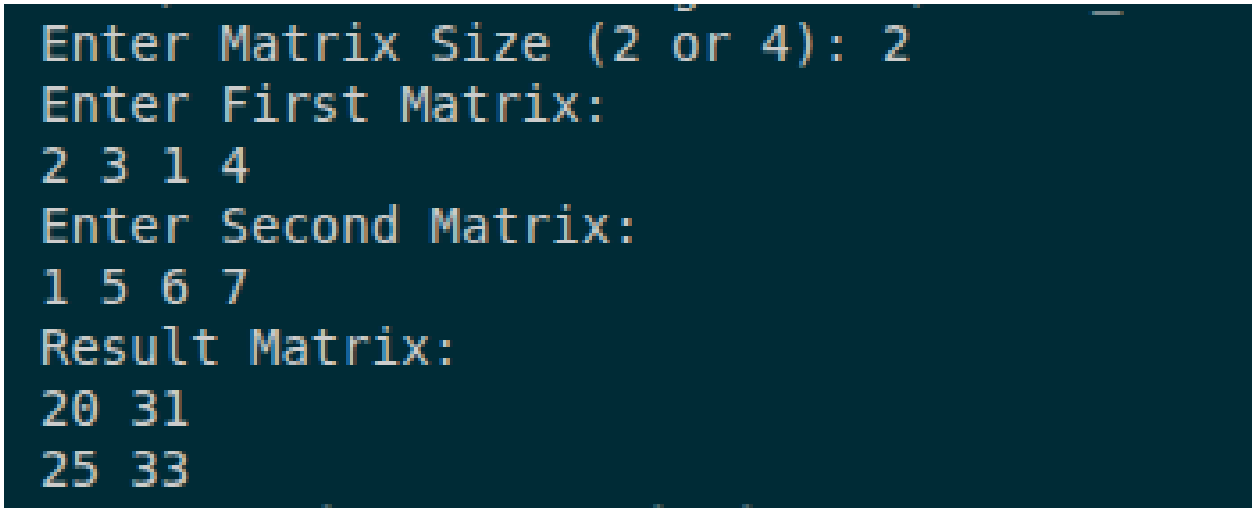
Problem

Write a program to implement Strassen's algorithm for matrix multiplication. Analyse how this approach is advantageous when compared to normal matrix multiplication.

Method

The matrices are divided into four submatrices. Strassen's method uses seven recursive matrix multiplications and additional matrix additions and subtractions. The resulting submatrices are then combined to form the final product. This implementation accepts matrix sizes 2 and 4.

Output:



Code:

```
/*
Program Name: Strassen's Matrix Multiplication
Program Description: Takes two square matrices and multiplies them using
                     Strassen's divide and conquer matrix multiplication
Program Creation Time: Sep 20th 4:50
Program Creator: Ashwath Raj | AP25110010712
*/

#include<iostream>
using namespace std;

void add(int A[4][4],int B[4][4],int C[4][4],int n){
    for(int i=0;i<n;i++)
        for(int j=0;j<n;j++)
            C[i][j]=A[i][j]+B[i][j];
}

void sub(int A[4][4],int B[4][4],int C[4][4],int n){
    for(int i=0;i<n;i++)
        for(int j=0;j<n;j++)
            C[i][j]=A[i][j]-B[i][j];
}

void strassen(int A[4][4],int B[4][4],int C[4][4],int n){
    if(n==1){
        C[0][0]=A[0][0]*B[0][0];
        return;
    }

    int m=n/2;

    int A11[4][4]={0},A12[4][4]={0},A21[4][4]={0},A22[4][4]={0};
    int B11[4][4]={0},B12[4][4]={0},B21[4][4]={0},B22[4][4]={0};
    int P1[4][4]={0},P2[4][4]={0},P3[4][4]={0},P4[4][4]={0};
    int P5[4][4]={0},P6[4][4]={0},P7[4][4]={0};
    int X[4][4]={0},Y[4][4]={0};

    for(int i=0;i<m;i++)
        for(int j=0;j<m;j++){
            A11[i][j]=A[i][j];
            A12[i][j]=A[i][j+m];
            A21[i][j]=A[i+m][j];
            A22[i][j]=A[i+m][j+m];

            B11[i][j]=B[i][j];
            B12[i][j]=B[i][j+m];
            B21[i][j]=B[i+m][j];
```

```
            B22[i][j]=B[i+m][j+m];
        }

    add(A11,A22,X,m);
    add(B11,B22,Y,m);
    strassen(X,Y,P1,m);

    add(A21,A22,X,m);
    strassen(X,B11,P2,m);

    sub(B12,B22,Y,m);
    strassen(A11,Y,P3,m);

    sub(B21,B11,Y,m);
    strassen(A22,Y,P4,m);

    add(A11,A12,X,m);
    strassen(X,B22,P5,m);

    sub(A21,A11,X,m);
    add(B11,B12,Y,m);
    strassen(X,Y,P6,m);

    sub(A12,A22,X,m);
    add(B21,B22,Y,m);
    strassen(X,Y,P7,m);

    for(int i=0;i<m;i++)
        for(int j=0;j<m;j++){
            C[i][j]=P1[i][j]+P4[i][j]-P5[i][j]+P7[i][j];
            C[i][j+m]=P3[i][j]+P5[i][j];
            C[i+m][j]=P2[i][j]+P4[i][j];
            C[i+m][j+m]=P1[i][j]-P2[i][j]+P3[i][j]+P6[i][j];
        }
}

int main(void){
    int n;

    cout<<"Enter Matrix Size (2 or 4): ";
    cin>>n;

    int A[4][4]={0},B[4][4]={0},C[4][4]={0};

    cout<<"Enter First Matrix:"<<endl;
    for(int i=0;i<n;i++)
        for(int j=0;j<n;j++)
            cin>>A[i][j];

    cout<<"Enter Second Matrix:"<<endl;
    for(int i=0;i<n;i++)
        for(int j=0;j<n;j++)
            cin>>B[i][j];

    strassen(A,B,C,n);

    cout<<"Result Matrix:"<<endl;
    for(int i=0;i<n;i++){
        for(int j=0;j<n;j++)
            cout<<C[i][j]<<" ";
        cout<<endl;
    }

    return 0;
}
```

Complexity

Normal Matrix Multiplication: O(n^3)
Strassen's Algorithm: O(n^log2(7)) approximately O(n^2.81)
Space Complexity: O(n^2)

Advantage over Normal Multiplication:
Normal matrix multiplication performs eight recursive submatrix multiplications when viewed through divide and conquer. Strassen's algorithm reduces this to seven multiplications by using additional matrix additions and subtractions. This changes the recurrence from T(n) = 8T(n/2) + O(n^2) to T(n) = 7T(n/2) + O(n^2), giving O(n^2.81) time. Therefore, Strassen's method has a better asymptotic multiplication complexity for sufficiently large matrices. For small matrices, the extra additions, subtractions, memory use, and implementation overhead can make normal multiplication preferable.

4. Greedy Approach: Fractional Knapsack

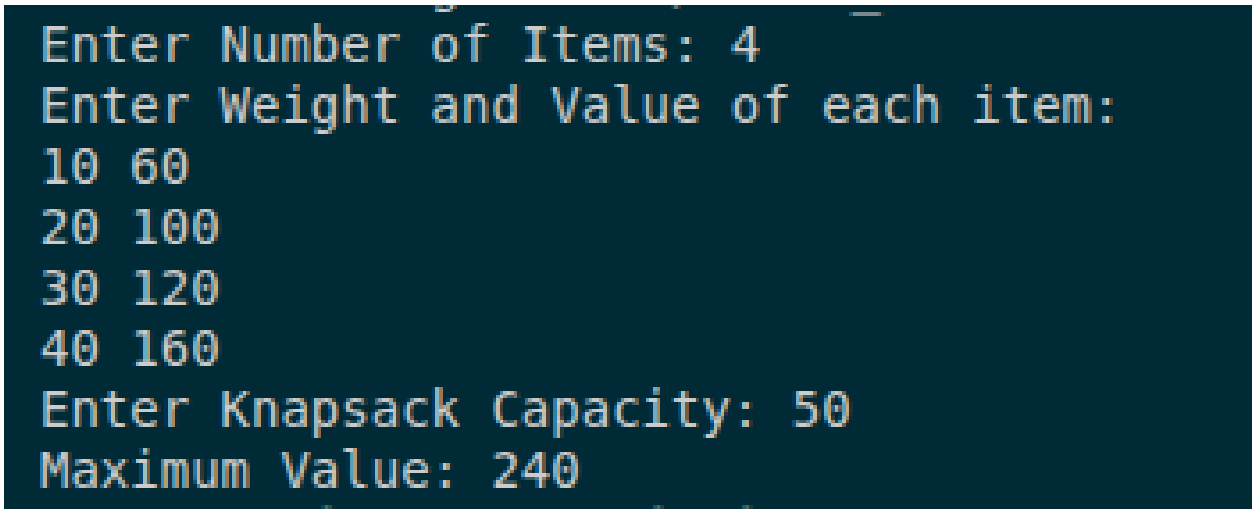
Problem

Write a program to solve the Fractional Knapsack problem using the greedy approach.

Method

The value-to-weight ratio of every item is calculated. Items are sorted in decreasing order of this ratio. The algorithm takes each complete item while possible and takes the required fraction of the next item when the remaining capacity is insufficient.

Output:



Code:

```
/*
Program Name: Fractional Knapsack Using Greedy Approach
Program Description: Takes the weight and value of items and finds the maximum
                     value that can be obtained using fractional knapsack
Program Creation Time: Sep 20th 5:00
Program Creator: Ashwath Raj | AP25110010712
*/

#include<iostream>
#include<algorithm>
using namespace std;

struct Item{
    int weight;
    int value;
    double ratio;
};

bool compare(Item a,Item b){
    return a.ratio>b.ratio;
}

double fractionalKnapsack(Item items[],int n,int capacity){
    sort(items,items+n,compare);

    double total=0;

    for(int i=0;i<n && capacity>0;i++){
        if(items[i].weight<=capacity){
            capacity-=items[i].weight;
            total+=items[i].value;
        }
        else{
            total+=(double)items[i].value/items[i].weight*capacity;
            capacity=0;
        }
    }

    return total;
}

int main(void){
    int n,capacity;

    cout<<"Enter Number of Items: ";
    cin>>n;

    Item items[n];
    cout<<"Enter Weight and Value of each item:"<<endl;
```

```
    for(int i=0;i<n;i++){
        cin>>items[i].weight>>items[i].value;
        items[i].ratio=(double)items[i].value/items[i].weight;
    }

    cout<<"Enter Knapsack Capacity: ";
    cin>>capacity;

    cout<<"Maximum Value: "<<fractionalKnapsack(items,n,capacity)<<endl;

    return 0;
}
```

Complexity

Time Complexity: O(n log n)
Space Complexity: O(n)

Time Complexity Derivation:
Calculating the value-to-weight ratios takes O(n). Sorting the items by ratio takes O(n log n), and selecting items takes O(n). Therefore, the overall time complexity is O(n log n). The greedy strategy is optimal for Fractional Knapsack because an item can be divided, so taking the available capacity with the highest value-to-weight ratio at every step produces the maximum total value.